

Screen Recognition & Window Control

Make Python see what's on screen, find a target, and move real windows.

Recap — where Day 18 left off

Day 18 built **autoclicker.py** — a global hotkey toggles clicking on/off, CPS is adjustable, hotkeys wired via decorator.

BEFORE

The clicker fires at fixed or cursor-follow coordinates. It is **blind** — it cannot tell whether the button it wants is actually there.

If the layout shifts, it clicks empty space.

AFTER (today)

Python **looks at the screen first**:
screenshot → find a target image → click its real center. Plus: move and focus windows.

Clicks land on what is actually visible.

Screenshots & reading pixels

What this is: the crudest way for Python to “see” — grab the screen as an image, then read color at a point.

```
import pyautogui

# grab the whole screen as an image
shot = pyautogui.screenshot()
shot.save("screen.png")

# read the RGB color at one point
px = pyautogui.pixel(100, 200)
print(px)

# is that point ~this color? (tolerance
# absorbs anti-aliasing wiggle)
hit = pyautogui.pixelMatchesColor(
    100, 200, (255,0,0), tolerance=10)
```

OUTPUT

```
(34, 87, 160)
False
```

Why it matters

▶ screenshot()

Returns a Pillow image of the live screen Python can inspect.

▶ pixel(x, y)

Reads the exact (R,G,B) at one coordinate.

Use it to detect a known state — a status light, a colored badge — not for general UI.

locateOnScreen — find a target

What this is: you save a small cropped PNG of a button; pyautogui scans the screen and tells you where it is.

```
import pyautogui

# search the screen for save_btn.png
box = pyautogui.locateOnScreen(
    "save_btn.png")
print(box) # Box(...) or None

# usually you just want the center point
pt = pyautogui.locateCenterOnScreen(
    "save_btn.png", confidence=0.8,
    grayscale=True)
print(pt) # Point(x, y) or None
```

OUTPUT

```
Box(left=812, top=44,
    width=68, height=24)
Point(x=846, y=56)
```

Anatomy

► returns a Box

left, top, width, height — or None if absent.

► confidence=0.8

Allows fuzzy matches (needs opencv).

grayscale=True is faster and more forgiving of minor color shifts.

Find, then click safely

What this is: the heart of screen_bot — locate the target, but always handle the case where it isn't on screen.

```
import pyautogui

def click_image(path, conf=0.85):
    pt = pyautogui.locateCenterOnScreen(
        path, confidence=conf)
    if pt is None:
        print("not found:", path)
        return False
    pyautogui.click(pt)  # click its center
    return True

click_image("save_btn.png")
```

Why the guard matters

► **locate returns None**

Not an exception — a missing image just gives None.

► **click(None) crashes**

So you must check before clicking.

► **returns a bool**

Lets you chain steps and bail cleanly.

TAKEAWAY

See first, then act. Never click a coordinate blindly
— confirm the target is visible.

pygetwindow — move & focus

What this is: a library that lists open windows and lets you focus, move, resize, or minimize them by title.

```
import pygetwindow as gw

# titles of every open window
print(gw.getAllTitles())

# grab a window by partial title
wins = gw.getWindowsWithTitle("Safari")
w = wins[0]

w.activate()      # bring to front
w.moveTo(0, 0)    # top-left
w.resizeTo(1280, 800)
w.minimize()
```

What it does

▶ enumerate windows

getAllTitles / getWindowsWithTitle.

▶ activate / move

Focus a window, reposition, resize, minimize.

⚠ WATCH OUT

Solid on Windows. On macOS support is partial and flaky — I'm not certain which ops work on current macOS. Verify; when something silently no-ops, fall back to AppleScript (next slide).

Window control via AppleScript

What this is: on macOS the dependable way to focus and size app windows is AppleScript, fired through your Day 16 bridge.

```
from os_bridge import run_applescript # Day 16

# focus an app — most reliable on mac
run_applescript(
    'tell app "Safari" to activate')

# move + resize the front window
run_applescript(''
tell application "Safari"
    set bounds of front window to
        {0, 0, 1280, 800}
end tell'')
```

Anatomy

▶ activate

Brings the app to the foreground (focus).

▶ set bounds

{x1, y1, x2, y2} — moves and sizes in one call.

▶ reuses os_bridge

One OS-control entry point from Day 16.

TAKEAWAY

AppleScript is mac-only and quoting-sensitive, but it's the path that actually works for windows.

Four ways to see & control

Method	What it does	Best for	Watch out
<code>pixel / matchesColor</code>	Reads RGB at a point	Detect a known color/state	Brittle; exact coords
<code>locateOnScreen</code>	Finds a cropped image	Clicking GUI buttons	Image must match; Retina scaling
<code>pygetwindow</code>	Lists/moves/resizes windows	Quick window ops	Flaky on macOS
<code>AppleScript bridge</code>	Focus/move/resize apps	Reliable mac windows	mac-only; quoting

screen_bot.py

Take a screenshot, find a cropped button image on screen, and click its real center. Bonus: a global hotkey that flips focus between two apps via AppleScript.

STRUCTURE

- Crop a button to a PNG
- Screenshot the screen
- Locate the PNG on it
- Click its center point

FUNCTIONS

- `click_image(path, conf)`
- `find_or_wait(path, timeout)`
→ retry locate in a loop
- Guard every None case

BEHAVIOR

- Runs end-to-end once
- Prints found / not found
- Bonus: cmd+shift hotkey
- toggles app focus

Things that will trip you up

Screen Recording permission

Until you grant Python/Terminal Screen Recording in System Settings, screenshots come back black and locate returns None. Separate from the Accessibility perm Day 17 needed.

Retina 2× scaling

On Retina Macs the screenshot is double the logical resolution, so locate coords can be 2× what click expects. I'm not certain pyautogui auto-corrects — print coords and halve if clicks land wrong.

confidence needs opencv

pip3 install opencv-python to use confidence=. Without it, matching is exact-pixel only and far more fragile.

grayscale=True

Faster matches and more tolerant of small color shifts. Cheap win — add it to most locate calls.

Python can now look and move

DAY 20

Morning launcher scaffold — schedule, clipboard, notifications, JSON routine loader.

DAY 21

Wire in the real stuff — Safari tabs + Spotify. Today's window/app control lands here.

CRUX TOOL-USE

“See the screen, act on it” is a core agent capability you'll reuse later.

Your scripts stopped being blind — they can see what's on screen and move real windows. Next, those actions get bundled into named routines.